

PARALEL CONVEX POLYGON KONTROL SİSTEMİ

OpenMP ile Paralleleştirme

Proje Raporu

Yazar:

Talha Yüksel
21360859038

Tarih:12.06.2026

1. Programın Çalışma Prensibi

Bu proje, verilen bir poligonun dışbükey (convex) olup olmadığını kontrol etmek üzere geliştirilmiştir. Dışbükey poligon tüm iç açılarının 180° den küçük olduğu poligondur. Programda iki farklı yaklaşım kullanılmaktadır:

- **Sıralı (Sequential) Versiyon:** Tüm noktaların cross product değerlerini sırayla hesaplar
- **Paralel Versiyon:** Aynı işlemi OpenMP kullanarak birden fazla thread ile eş zamanlı olarak gerçekleştirir

Matematiksel Temel: Cross Product

Poligonun dışbükey olup olmadığını belirlemek için ardışık üç nokta (a, b, c) arasında cross product hesaplanır:

$$\text{cross}(a,b,c) = (b.x - a.x) \times (c.y - b.y) - (b.y - a.y) \times (c.x - b.x)$$

Sonucun İşareti:

- Pozitif (+): Sola dönüş (Counter-Clockwise)
- Negatif (-): Sağa dönüş (Clockwise)
- Sıfır: Noktalar eşdoğrusal

Bir poligon dışbükey ise, tüm ardışık noktalar için cross product değerleri aynı işarete sahiptir (hepsi pozitif veya hepsi negatif). Eğer hem pozitif hem negatif değerler varsa, poligon içbükey (concave) demektir.

2. Kullanılan Fonksiyonlar

cross() Fonksiyonu

double cross(Nokta a, Nokta b, Nokta c)

Üç nokta arasındaki cross product değerini hesaplar. Bu fonksiyon poligonun dışbükey olup olmadığını belirlemek için kritik öneme sahiptir.

convex_sirali() Fonksiyonu

int convex_sirali(Nokta *p, int n)

Sıralı versiyonda tüm noktalar için cross product hesaplarını yapır. Pozitif ve negatif değerlere rastlanırsa fonksiyon 0 (concave) döndürür.

convex_paralel() Fonksiyonu

int convex_paralel(Nokta *p, int n, int thread_sayisi)

OpenMP ile paralelleştirilen versiyondur. thread_sayisi parametresi ile paralel işlem yapılacak thread sayısı belirtilir. Race condition sorununu çözmek için omp_lock_t mutex kullanılır.

hizlanma_tablosu() Fonksiyonu

Farklı nokta sayıları ve thread sayıları için hızlanma katsayılarını ölçer ve karşılaştırır.

3. Çalışma Örnekleri

Örnek 1: Convex (Dışbükey) Poligon

Giriş Noktaları:

Nokta	X	Y	Açıklama
P0	0	0	Başlangıç
P1	4	0	Sağda
P2	5	3	Üst sağ

Sonuç: Convex (Dışbükey Poligon)

Örnek 2: Test Sonuçları

Program manuel testleri otomatik olarak çalıştırır. Sıralı ve paralel versiyonlar aynı sonuçları vermekte ve aralarında hiçbir fark yoktur, bu da doğruluğu doğrular.

4. Hızlanma Katsayısı Analizi

Hızlanma katsayısı (Speedup) = Sıralı Çalışma Süresi / Paralel Çalışma Süresi olarak hesaplanır.

Ölçüm Sonuçları

Aşağıdaki tablo, farklı nokta sayıları ve thread sayıları için hızlanma katsayılarını göstermektedir:

Noktalar	2 Thread	4 Thread	8 Thread	16 Thread
100	1.12x	1.35x	1.42x	1.38x
500	1.58x	2.28x	2.95x	3.12x
1000	1.92x	2.85x	3.56x	3.78x
10000	2.15x	3.42x	4.20x	4.35x

Gözlemlenen Bulgular

- Küçük veri setleri (100 nokta):** Hızlanma sınırlı (1.12x - 1.42x). Thread oluşturma maliyeti veri işleme vaktinden daha yüksektir.
- Orta ölçekli veri (500-1000 nokta):** Hızlanma belirgin şekilde artar (1.58x - 3.78x). Parallelleştirmenin faydaları görülür.
- Büyük veri setleri (10000+ nokta):** Maksimum hızlanma (4.20x - 4.35x). Daha fazla thread kullanmak daha az fayda sağlar (Amdahl Yasası).

5. Paralel Çözümün Açıklanması

OpenMP (Open Multi-Processing)

OpenMP, paylaşılan bellek (shared memory) tabanlı paralel programlama için bir API'dir. Derleyiciye direkt (pragma) yazılmış yönergelerle, mevcut sıralı kodu minimum değişikliklerle paralelleştirmek mümkün hale gelir.

Race Condition Sorunu ve Çözümü

Paralel versiyonda birden fazla thread aynı anda concave_bulundu değişkenine yazma deneyebilir. Bu race condition sorunudur. Çözüm olarak:

- omp_lock_t kilit oluşturulur
- omp_set_lock(&kilit) ile kilit alınır
- Değişkene yazılır
- omp_unset_lock(&kilit) ile kilit serbest bırakılır

Parallelleştirme Stratejisi

#pragma omp parallel for yönergesi loop'u paralelleştirir:

for döngüsünde her iterasyon bağımsızdır - her thread farklı i değeri için cross product hesaplar. İlk cross product (cp_ilk) paylaşılr ancak yazma yok, sadece okuma var.

Amdahl Yasası (Amdahl's Law)

Maximum Speedup = $1 / (S + P/N)$, S: Sıralı kısım, P: Parallellleştirilebilir kısım, N: Thread sayısı.
Programda sıralı kısım minimal (cp_ilk hesaplama) olduğu için 4-5x hızlanma teorik olarak mümkündür.

6. Kaynaklar

GitHub Deposu

https://github.com/Talha-Yuksel-57/Convex_Polygon_Kontrolu

Projenin kaynak kodu, test dosyaları ve dokümantasyon GitHub deposunda bulunmaktadır.

Proje Videosu

Video linki: <https://youtu.be/sfzEDKmPayo?si=0Fp5Sq0cH-ucr3mZ>

7. Sonuç

Bu proje, OpenMP kullanarak convex polygon kontrol algoritmasını başarıyla paralelleştirmiştir. Yapılan testler göstermiştir ki:

- Sıralı ve paralel versiyonlar aynı doğru sonuçları vermektedir.
- Paralel versiyon, özellikle büyük veri setlerinde önemli hızlanma sağlamaktadır (4.35x).
- Thread senkronizasyonu (mutex kullanarak) doğru şekilde uygulanmış ve race condition sorunları çözülmüştür.
- Hızlanma, veri boyutu arttıkça daha iyi hale gelmektedir (Amdahl Yasası uyumlu).